

Middleware, Dependencies & Sessions

Web Design and Programming

M.Sc. Le Duy Khanh

Faculty of Data Science and Artificial Intelligence

Session Outline

1. HTTP Is Stateless
2. Middleware
3. Dependency Injection
4. Common Dependency Patterns
5. Cookies
6. Sessions
7. Flash Messages
8. Strategy for DS & AI Students
9. Worked Example & Labs
10. Review & Summary

Learning Objectives

By the end of this session, you will be able to:

1. Explain why HTTP is **stateless** and why that forces cookies/sessions.
2. Write **middleware** that runs before/after every path operation.
3. Write dependency functions and inject them with `Depends()`.
4. Write a `yield`-based dependency for resources needing cleanup.
5. Read/set **cookies**; explain what `HttpOnly` protects against.
6. Use `SessionMiddleware` for signed session data.
7. Explain the flash-message pattern.

Where We Are

- Week 7: full CRUD endpoints, validated input, shaped output.
- This week: the plumbing *around* endpoints – middleware, dependency injection, cookies/sessions.
- Central question: HTTP forgets everything between requests – how do we make it remember a visitor?

HTTP Is Stateless

HTTP Is a Stateless Protocol

- Every request is handled **independently** – the server does not remember the previous one.
- To recognise "the same visitor" across requests, the client must resend identifying information with **every** request:
  - A **cookie** (+ server-side session data) – this week.
  - A **token** (sent in a header) – Week 11.
- Cause and effect: stateless protocol → need to correlate requests → cookies/sessions/tokens.

The Problem: Recognizing Returning Visitors

- Example: a shopping-cart app.
  1. Request 1: $\mathrm{POST}\ /\mathrm{cart/add}$ – add a book. Server processes it, then forgets everything.
  2. Request 2: $\mathrm{GET}\ /\mathrm{cart}$ – "show my cart." Which cart? The server has **no idea who is asking**.
- Without extra information travelling with *each* request, every request looks like it's from a brand-new stranger.
- **Why we need cookies/sessions:** something must ride along on every request saying "this is the same visitor as before."

Middleware

What Is Middleware?

- **Definition:** a function that sits *between* the raw request and your path operation – it can inspect/modify the request before your code runs, and the response before it's sent back.
- Applies to **every request**, regardless of which route was called – you don't opt in per route.
- Analogy: airport security – everyone passes through the same checkpoint before reaching their gate (the path operation).
- Contrast with a **dependency** (Section 3): opt-in per route, and aware of route-specific parameters.

Why Do We Need Middleware?

```python
# Without middleware: the same logic copy-pasted into every route
@app.get("/items")
def list_items():
    start = time.perf_counter()
    result = ...
    print(f"took {time.perf_counter() - start:.3f}s")   # repeated everywhere
    return result

@app.get("/users")
def list_users():
    start = time.perf_counter()      # copy-pasted again, easy to forget
    ...
```

- **Cross-cutting concerns** – logging, timing, auth checks, compression – apply to *many* routes, not just one.
- Middleware centralizes this logic in **one place**, applied automatically to **all** requests: no copy-paste, nothing to forget.

Middleware Wraps Every Request

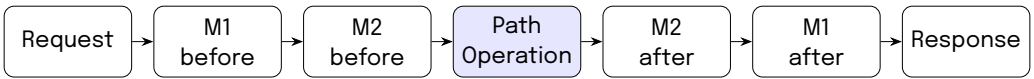
```python
import time
from fastapi import FastAPI, Request

app = FastAPI()

@app.middleware("http")
async def add_process_time(request: Request, call_next):
    start = time.perf_counter()
    response = await call_next(request)
    response.headers["X-Process-Time"] = str(time.perf_counter() - start)
    return response
```

- Code *before* `call_next()` runs on the way in; code *after* runs on the way out.
- **Registration order matters** – middleware nests like layers of an onion.

The Onion Model: Execution Order



- Registered order: $\mathrm{add_middleware(M1)}$ then $\mathrm{add_middleware(M2)}$.
- M1 (registered first) becomes the **outermost** layer; M2 (registered later) sits **closer** to the route.
- Rule of thumb: the last middleware added runs its “before” code last, but its “after” code **first**.

Built-in Middleware & CORS

```python
from fastapi.middleware.cors import CORSMiddleware

app.add_middleware(
    CORSMiddleware,
    allow_origins=["https://ui.example.com"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

- `CORSMiddleware`: let a different-origin frontend call this API.
- `GZipMiddleware`: compress responses. `SessionMiddleware`: signed session cookie (Section 6).
- Origin = protocol + domain + port; browsers block cross-origin calls unless CORS allows them.

More Built-in Middleware

Middleware	What it solves
CORSMiddleware	Allow a different-origin frontend to call the API
GZipMiddleware	Compress large responses automatically
TrustedHostMiddleware	Reject requests with a forged Host header
HTTPSRedirectMiddleware	Force all traffic onto HTTPS
SessionMiddleware	Signed session cookie (covered later today)

- Each one solves a **cross-cutting concern** you would otherwise reimplement per route.
- You can also write **custom** middleware for concerns specific to your app.

Custom Middleware: Global Error Handling

```python
from fastapi.responses import JSONResponse

@app.middleware("http")
async def catch_exceptions(request: Request, call_next):
    try:
        return await call_next(request)
    except Exception as exc:
        print(f"Unhandled error on {request.url.path}: {exc}")
        return JSONResponse(status_code=500,
                            content={"detail": "Internal server error"})
```

- Catches exceptions from **any** route in one place – no try/except needed in every path operation.
- Without it: either duplicate try/except everywhere, or let the app crash with an unfriendly trace.

Middleware vs Dependency: When to Use Which

	Middleware	**Dependency**
Scope	Every request, whole app	Opt-in, per route/router
Sees path params?	No	Yes
Can short-circuit	Yes (skip $\mathrm{call_next}$)	Yes (raise $\mathrm{HTTPException}$)
Typical use	Logging, CORS, timing, compression	Auth, DB session, pagination
Shows up in $/\mathrm{docs}$?	No	Yes (as parameters)

- Rule of thumb: **global & route-agnostic** $\rightarrow$ middleware; **needs route context or should appear in docs** $\rightarrow$ dependency.

Dependency Injection

A Dependency Is Just a Callable

```python
def pagination(skip: int = 0, limit: int = 10):
    return {"skip": skip, "limit": limit}

@app.get("/items")
def list_items(page: dict = Depends(pagination)):
    ...
```

- Depends(pagination): FastAPI calls pagination() first, injects its return value as page.
- You already used dependency-like mechanisms: Path, Query, Body (Week 7).
- Benefits: reuse, testability (override in tests), /docs visibility, avoids hidden coupling.

Dependency Scope

Scope	How
Single route	`arg: T = Depends(dep_func)`
Route (no return needed)	`@app.get(url, dependencies=[Depends(dep_func)])`
Router (group)	`APIRouter(dependencies=[Depends(dep_func)])`
Whole app	`FastAPI(dependencies=[Depends(dep_func)])`

- Use the decorator/router/app form for checks whose return value isn't needed (e.g. "reject if unauthenticated").

Common Dependency Patterns

Authentication Dependency

```python
from fastapi import Header, HTTPException

def verify_api_key(x_api_key: str = Header(...)):
    if x_api_key != "expected-secret":
        raise HTTPException(status_code=401, detail="Invalid API key")
    return x_api_key

@app.get("/secure-data", dependencies=[Depends(verify_api_key)])
def secure_data():
    return {"ok": True}
```

- Reads a header/cookie, validates it, returns the user or raises HTTPException(401).

yield **Dependencies – Setup & Cleanup**

```python
def get_session():
    with Session(engine) as session:
        yield session   # runs after the response is sent -> auto-closes

@app.get("/items")
def list_items(session: Session = Depends(get_session)):
    return session.exec(select(Item)).all()
```

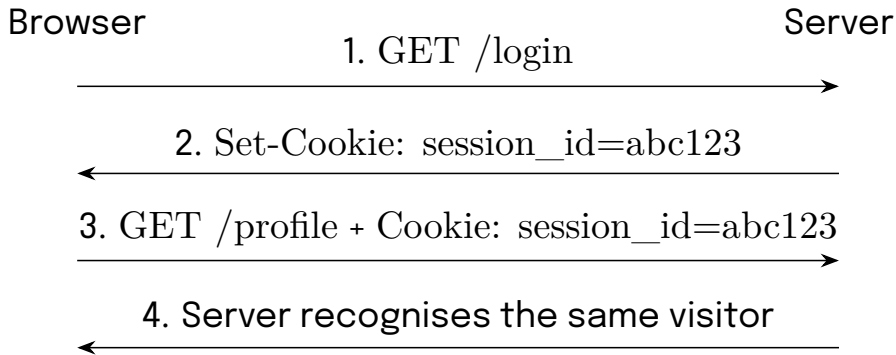
- Code *before* yield = setup; code *after* = teardown, guaranteed even on exceptions.
- Standard pattern for a DB session – reused heavily from Week 9 onward.
- **Sub-dependencies:** a dependency can itself Depends() on another, forming a chain.

Cookies

What Is a Cookie?

- **Definition:** a small piece of data (a name/value pair, $\leq 4\,\text{KB}$) that a server asks the browser to store, and that the browser **automatically resends** on every later request to the same site.
- Invented specifically to solve statelessness – it “glues” independent requests into one visit.
- **Why we need it:** without something the client resends by itself, the server has no way to link request #2 to request #1 from the same visitor.
- Stored client-side (in the browser), scoped to a domain and path.

How Cookies Work



- Step 2 happens **once**; the browser then attaches the cookie automatically on every later request – no client-side app code required.

Anatomy of a Cookie

Attribute	Meaning
Name=Value	The actual data, e.g. session_id=abc123
Domain	Which host(s) receive this cookie
Path	Which URL paths receive this cookie
Expires / Max-Age	When the browser deletes the cookie
Secure	Only sent over HTTPS
HttpOnly	Not readable by client-side JavaScript
SameSite	Controls cross-site sending (CSRF defense)

- Set via the Set-Cookie response header; the browser enforces Domain/Path/Expires for you.

Session Cookies vs Persistent Cookies

	Session cookie	**Persistent cookie**
Lifetime	Until the browser closes	Until `Expires`/`Max-Age`
Set by	No `Expires` attribute	`Expires` or `Max-Age` given
Typical use	Login state for one visit	"Remember me," preferences

- Naming clash to watch for: a "session cookie" (a *lifetime* category) is **not** the same thing as the server-side "session" we cover next – a session cookie can be used to *carry* a session ID.

Reading & Writing Cookies

```python
from fastapi import Cookie, Response

@app.get("/visits")
def visits(session_id: str | None = Cookie(default=None)):
    return {"session_id": session_id}

@app.get("/login")
def login(response: Response):
    response.set_cookie(key="session_id", value="abc123",
                        httponly=True, samesite="lax")
    return {"status": "logged in"}
```

- HttpOnly: blocks client-side JavaScript from reading the cookie (limits XSS damage).
- SameSite: restricts cross-site sending – early CSRF defense. Deep dive: Week 11.

Cookie Security Attributes in Depth

- HttpOnly – blocks document.cookie in JavaScript. Why: if an attacker injects a script (XSS), they still can't steal the cookie.
- Secure – the browser only sends the cookie over HTTPS. Why: prevents an eavesdropper on the network from reading it in plaintext.
- SameSite (Strict/Lax/None) – controls whether the cookie is sent on cross-site requests. Why: an early defense against CSRF (a malicious site tricking the browser into using your cookie).
- None of these **encrypt** the value – they control *who can read/send* the cookie, not its content.
- Full attack/defense detail: Week 11.

Limitations of Cookies

- **Size cap:** ≈ 4 KB per cookie – can't store a shopping cart or a full user profile.
- **Sent on every request:** large or numerous cookies add overhead to *every* request/response.
- **Client-controlled:** the browser (or user, or malware) can edit, delete, or refuse cookies – **never trust raw cookie content**.
- **Not private:** unless encrypted, the value is visible to whoever holds the cookie – `HttpOnly` only blocks JavaScript, not the user.
- **Conclusion – why we need sessions:** keep only a small opaque identifier in the cookie; store the real data, and the trust, on the **server**.

Sessions

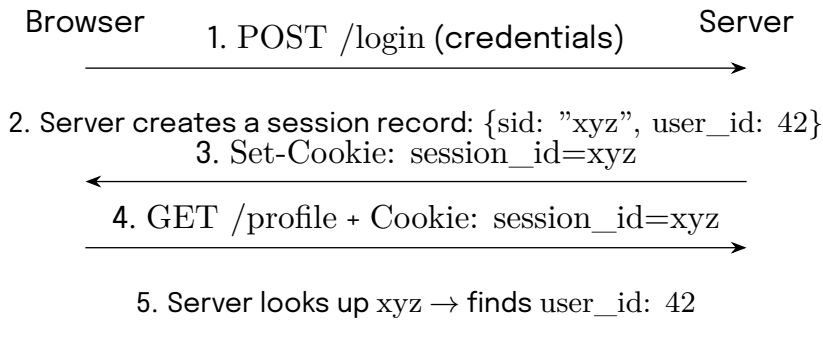
What Is a Session?

- **Definition:** server-side storage that holds data **about one visitor** across multiple requests, keyed by an identifier the server hands to the client.
- The client only ever holds the **identifier** (session ID); the actual data lives on the server.
- Analogy: a coat-check ticket – you carry a numbered ticket (session ID); the coat (your data) stays at the counter (server).

Why Do We Need Sessions?

- Cookies alone are small (≤ 4 KB), client-visible, and client-editable – unsafe or insufficient for real application state.
- Sessions fix this by keeping data **server-side**, where the client can't read or tamper with it directly.
- Enables things a plain cookie can't do safely:
  - Store large or sensitive per-user data (cart contents, permissions).
  - **Instantly revoke** access – delete the server-side record; a signed cookie alone can't be "recalled."
  - Look up the same visitor's state from **any** server in a multi-server deployment, if the store is shared (e.g. Redis).

How Sessions Work



- The cookie carries only the **key**; step 5 is a server-side lookup (in-memory dict, DB, Redis, ...).

SessionMiddleware – **Signed Cookie Sessions**

```python
from starlette.middleware.sessions import SessionMiddleware

app.add_middleware(SessionMiddleware, secret_key="change-me")

@app.get("/set")
def set_session(request: Request):
    request.session["user_id"] = 42
    return {"status": "set"}
```

- "Signed" = tamper-evident, **not encrypted** – never put secrets directly in it.
- Good for small, non-sensitive state: a user ID, a preference, a flash message.

Client-side vs Server-side Sessions

	Client-side (signed cookie)	Server-side (session store)
Data lives in	The cookie itself (signed)	DB/Redis/memory; cookie holds only an ID
Size limit	≈ 4 KB	Effectively unlimited
Instant logout/revoke	Hard (must wait for expiry)	Easy – delete the server record
Server storage needed	No	Yes (shared store at scale)
Example	`SessionMiddleware`	`session_id` cookie + Redis/DB

- Choose based on sensitivity/size of the data and whether you need instant revocation.

Session Lifecycle: Timeout & Invalidation

- **Creation:** on login (or first visit), the server generates a new session ID/record.
- **Timeout:** sessions should **expire** after inactivity (e.g. 30 min) – limits damage if a session ID leaks.
- **Invalidation (logout):** delete the server-side record and/or clear the cookie – with server-side sessions this is **immediate**; with a pure signed-cookie session, the client must simply discard it.
- **Session fixation** (an attacker planting a known session ID): mitigated by issuing a **new** session ID after login – full treatment in Week 11.

Flash Messages

Flash Message / Transient State

- A message shown **exactly once**, on the very next request (typically after a redirect).
- Pattern:
  1. Store the message in $\mathrm{request.session}$ before redirecting.
  2. On the next request, read it and **remove** it (pop) so it doesn't show again.
- Common building block in server-rendered apps with forms; depends entirely on the session mechanism.

Strategy for DS & AI Students

Learning Strategy

- `Depends` ≈ **pytest fixture:** prepare what's needed, inject it, clean up after – same idea as a `yield` fixture.
- **Middleware ≈ an outer decorator:** logic that runs around a function without modifying it.
- **Stateless is *why* login exists:** a cookie/token must travel on *every* request – the server has no memory.
- **Cookie security at awareness level now:** `HttpOnly`/`SameSite` here; CSRF/XSS/session fixation in depth – Week 11.

Worked Example & Labs

Worked Example: Visit Counter

```python
@app.middleware("http")
async def log_requests(request: Request, call_next):
    start = time.perf_counter()
    response = await call_next(request)
    print(f"{request.method} {request.url.path} -> "
          f"{response.status_code} ({time.perf_counter()-start:.3f}s)")
    return response

@app.get("/visits")
def count_visits(response: Response, visits: str | None = Cookie(default=None)):
    count = int(visits) if visits else 0
    count += 1
    response.set_cookie(key="visits", value=str(count),
                        httponly=True, samesite="lax")
    return {"visits": count}
```

Lab 1: Middleware

Hands-on practice

- Write a middleware logging method, path, and processing time for every request.
- Enable $\mathrm{CORSMiddleware}$ for your frontend's origin.

Lab 2: Dependency

Hands-on practice

- Write $\text{pagination}(\text{skip}, \text{limit})$ and reuse it in ≥ 2 routes.
- Write verify_api_key reading header X-API-Key; return 401 if wrong.

Lab 3: Cookie/Session + Project Integration

Hands-on practice

- Build a visit-counter endpoint using a cookie.
- Add $\mathrm{Depends}(\mathrm{get_session})$ as a yield-dependency (reused in Week 9).
- Refactor your group project's duplicated logic into a shared dependency.

Review & Summary

Quick MCQ Review

1. “HTTP is stateless” means? **No memory between requests; each is independent**
2. When does an `http` middleware run? **Before *and* after every path operation**
3. Main purpose of `Depends`? **Reuse shared logic; inject it; easy to test/override**
4. When does code after `yield` in a dependency run? **After the response is sent – cleanup**
5. What does `HttpOnly` prevent? **Client-side JavaScript reading the cookie**
6. Which middleware allows a different-origin frontend? `CORSMiddleware`

Self-Check Checklist

- ☐ Can explain stateless HTTP and why cookies/sessions exist
- ☐ Can write and register middleware (before/after logic)
- ☐ Can write a dependency and inject it with `Depends()`
- ☐ Know the four dependency scopes
- ☐ Can write a `yield`-based cleanup dependency
- ☐ Can read/set cookies and explain `HttpOnly`
- ☐ Know the difference between signed-cookie and server-side sessions

Summary and Next Session

- **Stateless HTTP** is why cookies/sessions (and later tokens) exist – to recognise the same client across requests.
- **Middleware** wraps every path operation; registration order sets the nesting order.
- `Depends()` injects a dependency's return value; reuse logic at route/router/app scope.
- A `yield` **dependency** sets up before `yield` and tears down after – the DB-session pattern.
- **Cookies**: `Cookie()` to read, `set_cookie()` to write; `HttpOnly` blocks JS access.
- `SessionMiddleware`: signed (not encrypted) cookie session; server-side store is the scalable alternative.
- **Flash message**: one-time message via the session, read then removed.
- Next: deeper database integration building on `get_session` (Week 9).

Q&A

Questions & Answers